

Boencreate Sales App

A multi-device sales and inventory tracker with login, real-time sync, and a shared backend — built with plain HTML/CSS/JS on the frontend and Supabase (Postgres + Auth + Realtime) on the backend. No custom server to run or maintain.

Features

- **Login/signup** — email + password, so your data is private to your account
 - **Multi-device sync** — add a sale on your phone, it appears instantly on your laptop (and vice versa) via Supabase Realtime
 - **Dashboard** — this month's sales/profit, low-stock alerts, 6-month revenue chart, recent sales
 - **New Sale** — item picker, live profit/margin calculation, automatic stock deduction
 - **Inventory** — add/update/delete items, stock + reorder level, low-stock flags
 - **Reports** — filter by date range/item, totals, CSV export
-

Step-by-step setup

Step 1 — Create a Supabase project

1. Go to supabase.com and sign up (free tier is enough)
2. Click **New Project**, choose a name, database password, and region
3. Wait ~2 minutes for the project to provision

Step 2 — Create the database tables

1. In your Supabase project, go to **SQL Editor** → **New query**
2. Open `schema.sql` from this repo, paste its contents, and click **Run**
3. This creates the `items` and `sales` tables, turns on Row Level Security (so only logged-in users can read/write), and enables realtime updates on both tables

Step 3 — Configure authentication

1. In Supabase: **Authentication** → **Providers** — Email should already be enabled by default
2. **Authentication** → **Settings** — for quicker testing, you can turn **off** "Confirm email" so new accounts can sign in immediately without clicking an email link (turn it back on later if you want extra security for a production app)

Step 4 — Get your API keys

1. In Supabase: **Settings** → **API**
2. Copy the **Project URL** and the **anon public** key (NOT the `service_role` key — that one must never be exposed in frontend code)

Step 5 — Connect the frontend

1. Open `index.html` in this repo

2. Near the top of the `<script>` section, find:

```
const SUPABASE_URL =  
'YOUR_SUPABASE_PROJECT_URL';  
const SUPABASE_ANON_KEY =  
'YOUR_SUPABASE_ANON_KEY';
```

3. Replace both with the values from Step 4

The anon key is safe to expose in client-side code — it only grants what your Row Level Security policies allow (Step 2 restricts everything to logged-in users).

Step 6 — Test locally

```
python3 -m http.server 8000
```

Visit `http://localhost:8000`, create an account on the login screen, sign in, and try adding an item and a sale.

Step 7 — Push to GitHub

```
git init  
git add .  
git commit -m "Initial commit: multi-device sales app"  
git branch -M main  
git remote add origin  
https://github.com/YOUR_USERNAME/YOUR_REPO.git  
git push -u origin main
```

Step 8 — Deploy (pick one)

Netlify (recommended — free, auto-deploys on every push):

1. netlify.com → **Add new site** → **Import an existing project**
2. Connect your GitHub account, select this repo
3. Leave build settings blank (no build step needed) → **Deploy**
4. You'll get a live URL like `yourapp.netlify.app`

Vercel (same idea):

1. vercel.com → **New Project** → import the repo
2. No framework/build settings needed → **Deploy**

GitHub Pages:

1. Repo → **Settings** → **Pages**
2. Source: branch `main` , folder `/ (root)` → **Save**
3. Live at `https://YOUR_USERNAME.github.io/YOUR_REPO/`

Step 9 — Create your user accounts

Once deployed, open the live URL and use the signup form to create accounts for yourself and any staff who'll use it. Every account that signs up can see and edit the same shared data (items/sales aren't scoped per-user in this version — see "Multi-user permissions" below if you want to change that).

Multi-user permissions (optional next step)

Right now, `schema.sql` gives **every logged-in user full access** to all items and sales (simplest setup for a small team sharing one dataset). If you later want to restrict what

certain users can see or edit — e.g., staff can add sales but not delete inventory, or each user only sees their own records — that means adding an `owner` column and adjusting the RLS policies, or using Supabase's role-based policies. Worth revisiting once you know how many people will use it and what they each need to do.

Project files

File	Purpose
<code>index.html</code>	The entire app — HTML, CSS, and JS in one file
<code>schema.sql</code>	Database schema + security policies to run in Supabase
<code>README.md</code>	This file

Costs

Supabase free tier covers this comfortably for a small business (500MB database, 50,000 monthly active users, 2GB file storage, 2GB bandwidth). Netlify/Vercel free tiers are also generous for a low-traffic internal tool. You likely won't pay anything unless usage grows significantly.